C# Array Methods Reference Guide

Basic Array Creation and Initialization

csharp

```csharp
// Array declaration and initialization
int[] numbers = new int[5];                    // Creates array of 5 integers with default va
int[] predefinedNumbers = new int[] {1, 2, 3, 4, 5};  // Initialize with values
int[] shorthandNumbers = {1, 2, 3, 4, 5};      // Shorthand initialization
```

Array Properties

csharp

```csharp
int length = myArray.Length;      // Gets the total number of elements
int rank = myArray.Rank;          // Gets the number of dimensions
```

Array Class Static Methods

Creating and Initializing

csharp

```csharp
// Create an array
Array myArray = Array.CreateInstance(typeof(int), 5);

// Initialize all elements to a specific value
Array.Clear(myArray, 0, myArray.Length);

// Fill an array with identical values
int[] numbers = new int[5];
Array.Fill(numbers, 42);  // All elements become 42
```

Searching and Finding

csharp

```csharp
int[] numbers = {4, 7, 2, 9, 1, 5};

// Find the index of an element
int index = Array.IndexOf(numbers, 9);  // Returns 3

// Find the last occurrence of an element
int lastIndex = Array.LastIndexOf(numbers, 5);  // Returns 5

// Find the first index that matches a condition
int firstMatch = Array.FindIndex(numbers, x => x > 5);  // Returns 1 (index of 7)

// Find the last index that matches a condition
int lastMatch = Array.FindLastIndex(numbers, x => x < 5);  // Returns 4 (index of 1)

// Find the first element that matches a condition
int item = Array.Find(numbers, x => x > 5);  // Returns 7

// Find the last element that matches a condition
int lastItem = Array.FindLast(numbers, x => x < 5);  // Returns 1

// Find all elements that match a condition
int[] matches = Array.FindAll(numbers, x => x > 5);  // Returns {7, 9}

// Check if any element satisfies a condition
bool anyGreaterThan8 = Array.Exists(numbers, x => x > 8);  // Returns true (9 exists)

// Check if all elements satisfy a condition
bool allPositive = Array.TrueForAll(numbers, x => x > 0);  // Returns true
```

Sorting and Modifying

csharp

```csharp
int[] numbers = {4, 7, 2, 9, 1, 5};

// Sort array
Array.Sort(numbers);  // numbers becomes {1, 2, 4, 5, 7, 9}

// Sort with custom comparison
Array.Sort(numbers, (a, b) => b.CompareTo(a));  // Descending order: {9, 7, 5, 4, 2, 1}

// Sort with a key selector
string[] names = {"Tom", "Dick", "Harry"};
Array.Sort(names, (a, b) => a.Length.CompareTo(b.Length));  // Sort by string length

// Reverse array elements
Array.Reverse(numbers);  // {9, 7, 5, 4, 2, 1}

// Reverse a portion of the array
Array.Reverse(numbers, 1, 3);  // Reverses 3 elements starting at index 1

// Copy entire array
int[] copy = new int[numbers.Length];
Array.Copy(numbers, copy, numbers.Length);

// Copy a portion of an array
int[] partialCopy = new int[3];
Array.Copy(numbers, 1, partialCopy, 0, 3);  // Copy 3 elements starting at index 1
```

Converting and Resizing

csharp

```csharp
// Convert all elements using a converter function
int[] numbers = {1, 2, 3, 4, 5};
string[] strNumbers = Array.ConvertAll(numbers, x => x.ToString());

// Resize an array
Array.Resize(ref numbers, 10);  // Resizes to 10 elements, filling new ones with default values

// Create a new array as a subset
int[] subset = numbers.Skip(2).Take(3).ToArray();  // Get 3 elements starting at index 2
```

Extension Methods (LINQ)

csharp

```
using System.Linq;

int[] numbers = {4, 7, 2, 9, 1, 5};

// Filtering
int[] greaterThan5 = numbers.Where(x => x > 5).ToArray();  // {7, 9}

// Mapping/Transforming
int[] doubled = numbers.Select(x => x * 2).ToArray();  // {8, 14, 4, 18, 2, 10}

// Sorting
int[] ascending = numbers.OrderBy(x => x).ToArray();  // {1, 2, 4, 5, 7, 9}
int[] descending = numbers.OrderByDescending(x => x).ToArray();  // {9, 7, 5, 4, 2, 1}

// Aggregation
int sum = numbers.Sum();  // 28
int min = numbers.Min();  // 1
int max = numbers.Max();  // 9
double avg = numbers.Average();  // 4.67

// Quantifiers
bool anyGreaterThan8 = numbers.Any(x => x > 8);  // true
bool allPositive = numbers.All(x => x > 0);  // true

// Element Operations
int first = numbers.First();  // 4
int firstOrDefault = numbers.FirstOrDefault(x => x > 10, -1);  // -1 (default)
int last = numbers.Last();  // 5
int single = numbers.Where(x => x == 9).Single();  // 9

// Set Operations
int[] another = {1, 5, 8, 10};
int[] union = numbers.Union(another).ToArray();  // {4, 7, 2, 9, 1, 5, 8, 10}
int[] intersection = numbers.Intersect(another).ToArray();  // {1, 5}
int[] except = numbers.Except(another).ToArray();  // {4, 7, 2, 9}
int[] distinct = numbers.Distinct().ToArray();  // Removes duplicates

// Partitioning
int[] skip = numbers.Skip(2).ToArray();  // {2, 9, 1, 5}
int[] take = numbers.Take(3).ToArray();  // {4, 7, 2}
```

```csharp
// Joining
string joined = string.Join(", ", numbers);  // "4, 7, 2, 9, 1, 5"
```

Multidimensional Arrays

csharp

```csharp
// 2D array
int[,] matrix = new int[3, 4];  // 3 rows, 4 columns
matrix[0, 0] = 1;  // First element

// 3D array
int[,,] cube = new int[3, 3, 3];
cube[0, 0, 0] = 1;  // First element

// Get length of specific dimension
int rows = matrix.GetLength(0);  // 3
int cols = matrix.GetLength(1);  // 4

// Iterate through a multidimensional array
for (int i = 0; i < matrix.GetLength(0); i++)
{
    for (int j = 0; j < matrix.GetLength(1); j++)
    {
        Console.Write(matrix[i, j] + " ");
    }
    Console.WriteLine();
}
```

Jagged Arrays (Array of Arrays)

csharp

```csharp
// Declare a jagged array
int[][] jagged = new int[3][];

// Initialize each inner array
jagged[0] = new int[] {1, 2, 3};
jagged[1] = new int[] {4, 5};
jagged[2] = new int[] {6, 7, 8, 9};

// Access elements
int value = jagged[2][3]; // 9

// Iterating through a jagged array
for (int i = 0; i < jagged.Length; i++)
{
    for (int j = 0; j < jagged[i].Length; j++)
    {
        Console.Write(jagged[i][j] + " ");
    }
    Console.WriteLine();
}
```

Array Conversion Methods

csharp

```csharp
// Array to List
int[] numbers = {1, 2, 3, 4, 5};
List<int> numberList = numbers.ToList();

// List to Array
List<string> stringList = new List<string> {"a", "b", "c"};
string[] stringArray = stringList.ToArray();

// Array to Dictionary
Dictionary<int, string> dict = new Dictionary<int, string>();
for (int i = 0; i < numbers.Length; i++)
{
    dict.Add(i, numbers[i].ToString());
}

// Array to HashSet (removes duplicates)
int[] withDuplicates = {1, 2, 2, 3, 4, 4, 5};
HashSet<int> uniqueSet = new HashSet<int>(withDuplicates);
```

Manipulating Arrays with Span<T> (C# 7.2+)

csharp

```csharp
// Create a span over an array
int[] numbers = {1, 2, 3, 4, 5};
Span<int> span = numbers;
Span<int> slice = span.Slice(1, 3);  // {2, 3, 4}

// Modify through the span
slice[0] = 22;  // numbers is now {1, 22, 3, 4, 5}

// Check if span contains a value
bool hasValue = span.Contains(22);  // true

// Find index of a value
int index = span.IndexOf(3);  // 2

// Fill a span with a value
slice.Fill(50);  // numbers becomes {1, 50, 50, 50, 5}

// Copy to another span
Span<int> destination = new int[3];
slice.CopyTo(destination);  // destination is now {50, 50, 50}
```

Memory-Efficient Methods with ArrayPool<T> (C# 7+)

csharp

```csharp
using System.Buffers;

// Rent an array from the shared pool
ArrayPool<byte> pool = ArrayPool<byte>.Shared;
byte[] buffer = pool.Rent(1024);  // May return larger array than requested

try
{
    // Use the buffer...
    buffer[0] = 42;
}
finally
{
    // Return the array back to the pool
    pool.Return(buffer, clearArray: true);
}
```

Common Array Algorithms

csharp

```csharp
// Binary search (requires sorted array)
int[] sorted = {1, 3, 5, 7, 9, 11};
int index = Array.BinarySearch(sorted, 5);  // Returns 2

// Running sum / Prefix sum
int[] nums = {1, 2, 3, 4};
int[] runningSum = new int[nums.Length];
int sum = 0;
for (int i = 0; i < nums.Length; i++)
{
    sum += nums[i];
    runningSum[i] = sum;
} // Result: {1, 3, 6, 10}

// Find maximum subarray sum (Kadane's algorithm)
int[] numbers = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
int maxSoFar = numbers[0];
int maxEndingHere = numbers[0];
for (int i = 1; i < numbers.Length; i++)
{
    maxEndingHere = Math.Max(numbers[i], maxEndingHere + numbers[i]);
    maxSoFar = Math.Max(maxSoFar, maxEndingHere);
} // maxSoFar = 6 (subarray [4, -1, 2, 1])
```

Best Practices

- Consider using collection classes like `List<T>` for dynamic arrays
- Use LINQ for clear, expressive array operations
- Consider performance implications for large arrays:
  - Use `ArrayPool<T>` for frequent allocation/deallocation of large arrays
  - Use `Span<T>` for efficient memory manipulation without copying
  - Avoid excessive LINQ chaining for performance-critical code
- For multidimensional data, consider if a jagged array might be more efficient than a multidimensional array
- Use the most appropriate search method: `IndexOf` for unsorted arrays, `BinarySearch` for sorted arrays